



— TEST-TIME COMPUTE · 检索 · 冻结模型

搜索， 就是 **test-time compute**

不把模型做大，而是在推理时多花算力。
它能换来更高的精度，也能换来一个从没被训练过的新功能。

肖涵 Han Xiao

Elastic AI 副总裁 · Jina AI 创始人

@hxiao · hanxiao.io

今天的路线

一个观点，两个实验。

00

什么是 test-time compute

大模型靠「想得更久」变强。先把这件事说清楚。

01

搜索本来就是 test-time compute

你搭的每一条检索链路，都是在推理期花算力买相关性。

本次演讲的主线

02

拿它换精度

让 agent 通宵在一个冻结编码器上搜索程序，144 代。
结论出乎意料。

实验一

03

拿它换新功能

一个只会做检索的模型，被推理期计算改造成图像标注器。

实验二

两个实验都不改一个权重。区别只在于：算力买到的是更好的旧任务，还是一个新任务。

— 00 · 概念

Test-time compute: 在推理时多花算力，换一个更好的答案。

不去训练一个更大的模型，而是让现有模型**对每个请求做更多的工作**，沿着一条平滑的「成本—质量」曲线往上走。

- | **BEST-OF-N** 采样很多个候选，挑最好的那个。
- | **SELF-CONSISTENCY** 对多条思维链做多数投票。
- | **VERIFIER SEARCH** 花推理算力，爬一条 Pareto 曲线。

让一个机器人在一手扑克上多想 **20 秒**，带来的收益，相当于把模型放大 **10 万倍**、并且多训练 10 万倍的时间。

Noam Brown, OpenAI, 2024

但这一直被讲成大模型的故事。

通常的说法

test-time compute 属于**大型推理模型**：模型要够大，才有「多想一会儿」这个东西可想。小模型没有这个待遇。

但检索这一侧呢

向量模型只有几百 M 到 1B，一次前向就出一个向量。它有没有「多想一会儿」的空间？

这是今天要回答的问题

先别急着回答小模型行不行。先看清楚一件事：**搜索这件事本身，早就是 test-time compute 了。**

01 | 搜索就是 test-time compute

你搭的每一条检索链路，都是在推理期用算力买相关性

— 换个角度

你早就在做 test-time compute，只是没这么叫它。

你把训练好的 embedding、reranker、多向量检索、查询扩展，在**推理期串成一条链路**，去把相关性一点点榨出来。你并没有换一个更大的模型，你是在测试时**装配了更多的搜索**。

不 scale 的搜索

一次关键词匹配。便宜，但大概率不够好。

scale 之后的搜索

embed → 召回 → rerank → 扩展 → 融合。**更多推理，换更多相关性。**

这就是一条 **test-time compute 曲线**

所以真正的问题不是「模型够不够大」，而是**你在推理时装配了多少条链路，以及这笔算力到底划不划算。**

一个最小的例子

从一次 cosine，到 late interaction，中间全是算力。

单向量 COSINE



$$\cos(q, d)$$

一篇文档一个向量

冻结基线

句子级 MAXSIM

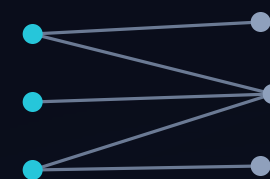


$$\max_s \cos(q, s)$$

同一个编码器，文档切成句子向量

这一段就是 **TEST-TIME COMPUTE**

COLBERT 多向量



$$\sum_i \max_j \cos(q_i, d_j)$$

每个 query token 对全部 doc token

取 max

需要专门的多向量模型

中间那一列很关键：同一个冻结模型，只是在推理时多算了几遍，就爬到了原本要换模型才有的位置。

— 本场枢纽

那么，这笔推理算力到底能换到什么？

01

换精度：同一个任务，做得更好

任务不变，还是检索。我让一个 agent 通宵在冻结编码器上搜索程序，用 144 代去逼这条曲线的上限。

看起来它成功了。然后在没见过的数据上翻车了。

实验一 · 接下来 15 分钟

02

换新功能：一个它从没学过的任务

同样是冻结的向量模型，这次不求它检索得更准，而是要它做一件训练时根本不存在的事：给图片打开放词表的标签。

这一次，算力真的买到了东西。

实验二 · 之后 12 分钟

两个实验，同一个问题的两面：推理期算力的边界在哪里，什么样的算力才真的算数。

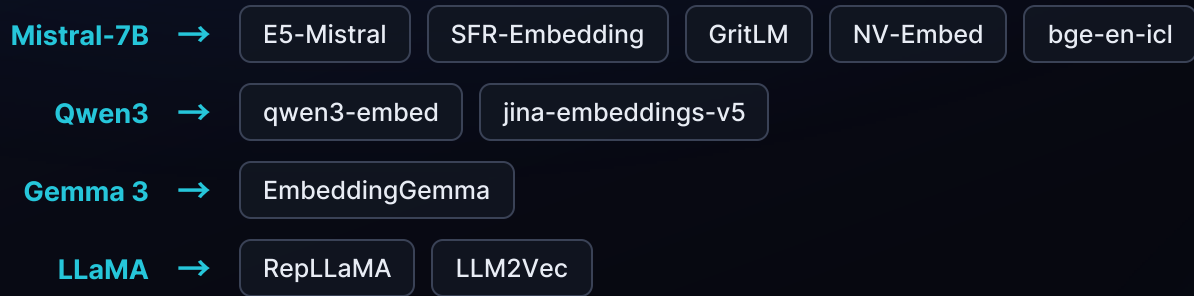
02 | 实验一 · 用算力换精度

让 agent 通宵在一个冻结编码器上搜索程序：144 代，12 个 Pareto 最优解，
和一个反转

— 先破一个偏见

「小模型没法靠推理变强」——可它们本来就是大模型蒸出来的。

通常认为 test-time compute 属于大型推理模型，小模型没有提升空间。但今天的向量模型，**包括那些几百 M 能直接部署的**，几乎全是从 LLM 骨干蒸馏或改造而来：



从大语言模型到小型可部署编码器，如今是主流配方，不是例外。

如果 test-time compute 住在 LLM 的表示空间里，这些蒸馏出来的编码器**理应继承它**。真的继承了吗？

问题

一个冻结的单向量编码器，只靠推理能提升多少？

不重训

不引入辅助模型

不学任何新参数

只有一个冻结编码器 API

最近这些检索侧的 test-time 方法，**每一个都至少违反一条**：

| **HYDE / QUERY2DOC** 查询路径里塞了一个外部 LLM。

| **GQR** 需要第二个检索器来做监督。

| **METAEMBED** 在部署时训练了额外参数。

这三条我们全禁掉，然后问：**在这么严的约束下，还剩下什么？**

方法

Autoresearch: 让 agent 自己去爬这座山。

agent 自己跑完整个研究循环：**改一个文件** → **跑一次固定预算的短实验** → **指标涨了就留下，没涨就回滚**。然后重复，通宵。本质上就是爬山法，只不过变异算子换成了一个 LLM。



整个过程我没有手写任何一个 Python 程序。144 代跑下来，全是 agent 自己写的。

这本身也是 test-time compute，只不过花在了「**搜索方法**」这一层，而不是「搜索文档」那一层。

循环里的四件套

提议者写程序，评估器打分，记忆决定下一步。



记忆回流：每一代都读得到前面所有代的成败与教训

239M

发现阶段用的编码器 jina-embeddings-v5-nano，小而快，循环转得动

14

MMTEB Tier-1 任务：法律、金融、长文档、通用

144

程序总数，每一代一个，全部记录 Δ、成本、父代、教训

— 先把成本定义清楚

成本只有一个数：多跑了几次编码器前向。

$$c = 1 + \frac{\text{额外的前向次数}}{\text{基线前向次数}}$$

$c = 1$ · 零额外前向

SoftCentroid：把已经算出来的 top-k 文档向量求平均，混进 query 里，再打一次分。

$$q' = q + \text{mean}(d_emb[\text{top-k}])$$

纯粹是在已有向量上做代数，一次新的前向都没有。

$c > 1$ · 真的多跑了模型

把文档切成句子重新编码、把 query 改写后重新编码、多轮反馈再编码——**每一次都要模型真的再跑一遍。**

本实验里最贵的程序 $c = 14.7$ ，也就是十五倍的推理成本。

这个区分是整个实验的关键：**纯代数** 和 **真花算力**，最后的命运完全不同。

— 同一个循环，两套评分标准

一套鼓励花算力，一套只认能迁移的东西。

Compute rubric · 算力标准

只要**域内**成绩超过此前所有程序就收录。明确鼓励提议者去花更多推理算力。

→ 域内的天花板到底能爬多高？

Transfer rubric · 迁移标准

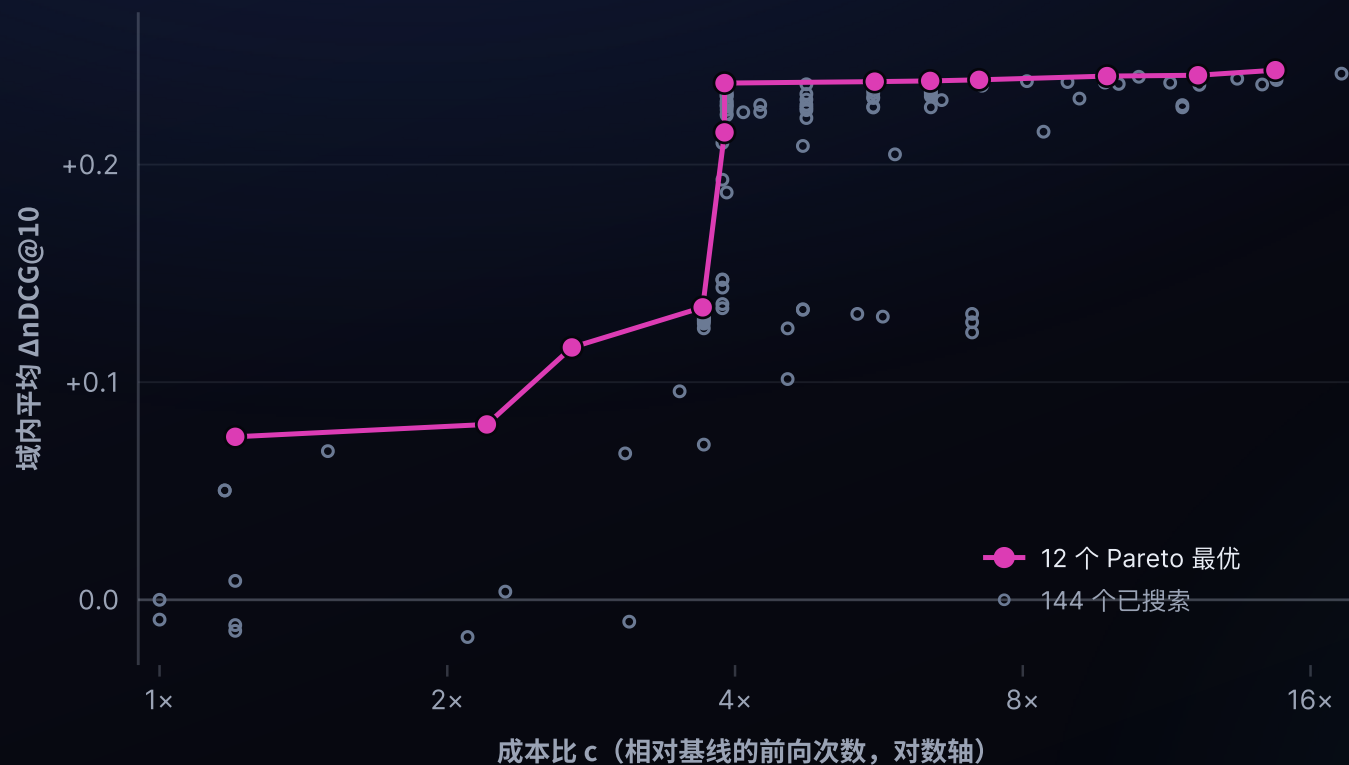
只有当验证集提升、且没有任何任务退步超过 0.05 才收录。花算力这件事本身不给任何奖励。

→ 什么东西能在域外站得住？

两次搜索都从未见过那 19 个留出任务，也从未见过留出的编码器家族。

算力买到了什么 · 域内

被要求花算力，搜索画出了一条漂亮的 Pareto 曲线。



144

144 代里搜出的程序总数

12

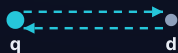
Pareto 最优解，成本从 $c = 1.2$ 一直到 14.7

域内平均 $\Delta nDCG@10$ 从 +0.07 爬到 +0.24。这看起来就是标准的 test-time compute scaling。

但这只是域内。
留出集的结果在下一页。

AGENT 写出了什么

算力前沿上的十二个程序。



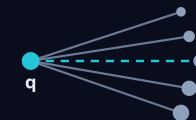
BidirZScore $c=1.2$



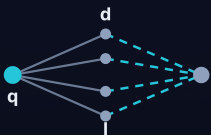
SentMaxSim $c=2.2$



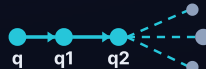
AdaptGranularity $c=2.7$



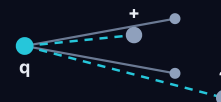
CoverageTriple $c=3.7$



LexicalHybridRRF $c=3.9$



CrossRoundRRF $c=3.9$



DiverseDualCtx $c=5.6$



ConsensusRocchio $c=6.4$



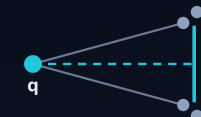
NegContrastive $c=7.2$



MomentumProg $c=9.8$



GraphCentrality $c=12.2$



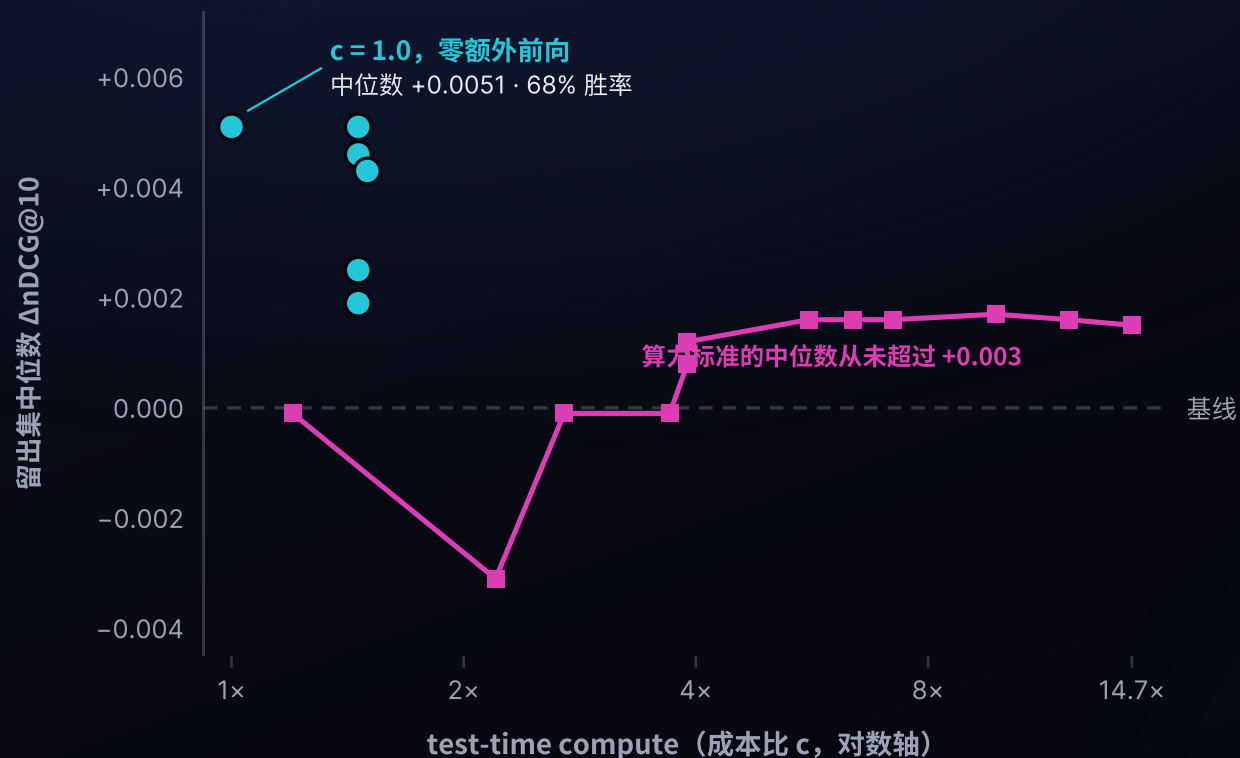
FisherStability $c=14.7$

每一个都是对同一批冻结向量的免训练重组。成本从左到右递增；而它们带来的提升，下一页会告诉你并没有跟着涨。

留出集上的真相

在没见过的编码器上，算力是平的。便宜的结构不是。

算力标准搜出来的程序 迁移标准搜出来的程序



14.7x

最贵的程序花的算力，被一个零额外前向的程序打平

-0.016

算力标准在留出集上的合并均值，低于基线

-0.98

它最差的单条 query 结果

迁移标准搜出的程序在 $c = 1.0$ 、零额外前向的情况下，就已经打赢了 $c = 14.7$ 那个最贵的算力程序。图里画的是中位数（对 -0.98 这种尾部更稳健），合并均值同样是负的。

逐格看这条算力前沿

算力大概帮到一半的格子，另一半直接塌了。



每一行是十二个程序之一（按算力成本 1.2× 到 14.7× 排序），每一列是十九个留出任务之一，四个编码器里有三个在发现阶段从未出现过。大约一半的格子是正的（912 格中 485 格），76 个「任务 × 编码器」组合里有 52 个至少在某个程序下有提升——但深粉色的格子能掉到 -0.98，所以合并均值仍然是负的 -0.016。

另一套标准

换个评分标准，搜出了六个便宜且能迁移的程序。

程序	c	中位数	胜率	均值	最差格
ZeroCost-Consensus	1.00	+0.0051	68.4%	+0.0148	-0.1070
Deca-Axis	1.46	+0.0051	68.4%	+0.0133	-0.0365
Dodeca-v077	1.46	+0.0046	66.7%	+0.0126	-0.0420
Transpose-Consensus	1.50	+0.0043	83.3%	+0.0219	0.0000
Cross-Axis	1.46	+0.0025	64.9%	+0.0097	-0.0240
Penta-Axis	1.46	+0.0019	59.6%	+0.0107	-0.0485

全部集中在 $c = 1.0$ 到 1.5 ，最好的那个（Transpose-Consensus, 83.3% 胜率）最差格是 0.0000 —— 一个任务都没被弄坏。
对比一下：算力标准那边最贵的程序花到了 $c = 14.7$ 。

它迁移到哪去了

提升最大的，恰恰是发现阶段从没见过编码器家族。

均值 中位数



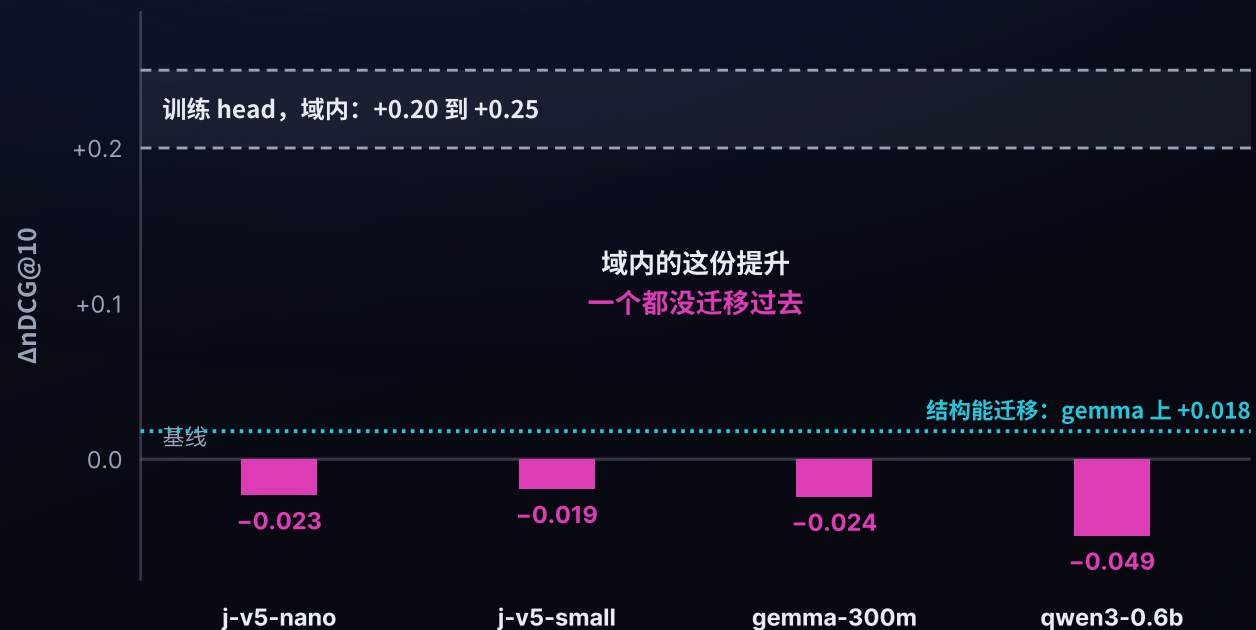
程序是在 **j-v5-nano** 上发现的。四个编码器上平均 $\Delta nDCG@10$ 全为正，而且在两个从未见过的家族上最大。

jina 家族上的中位数贴近零，说明提升集中在一条正的长尾里，不是全面普涨。

这说明它跟着的是通用的向量几何，而不是某个模型的怪癖。

一个必须回答的反问

那为什么不干脆训一个 head?



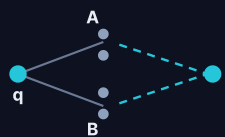
在同样这 14 个发现任务上训练, 线性 head、低秩 head、MLP head 都能把**域内**检索提升 +0.20 到 +0.25 $\Delta nDCG@10$ 。

但在**每一个**留出编码器上, 它们都掉到基线以下。

同样的数据预算, **加参数不迁移**,
重组已有的几何结构才迁移。

它到底找到了什么

反复出现的结构，是经典 IR 在向量空间里被重新推导了一遍。



Reciprocal Rank Fusion

REDISCOVERED, NOT SEEDED

fuse two rankings into one consensus



Fisher Linear Discriminant

REDISCOVERED, NOT SEEDED

the axis that best separates relevant from not



Rocchio Pseudo-Relevance Feedback

OPERATIONALIZED FROM A SEEDED IDEA

pull the query toward the relevant centroid



Sentence-level MaxSim

OPERATIONALIZED FROM A SEEDED IDEA

score the best sentence, not the mean

agent 没有被喂过这些名字。它从零把它们重新发明了出来——因为这些结构本来就在几何里。

用算力换精度，这条路撞墙了。

01 域内，它完美复现了 test-time compute scaling

144 代、12 个 Pareto 最优解、 $\Delta nDCG@10$ 从 +0.07 爬到 +0.24。曲线漂亮得可以直接放进论文。

02 域外，这条曲线整个消失了

合并均值 -0.016，最差单格 -0.98。花 14.7 倍算力的程序，打不过一个零额外前向的程序。

03 留下来的是结构，不是算力

六个能迁移的程序全在 $c = 1.0$ 到 1.5，而且在没见过的编码器上收益最大。它们是 RRF、Fisher 判别、Rocchio 反馈的再发现。

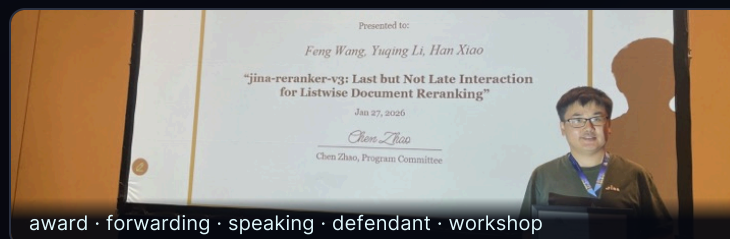
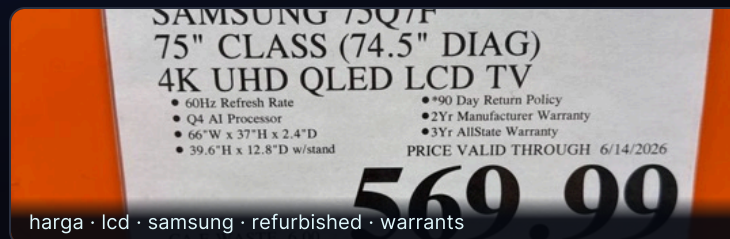
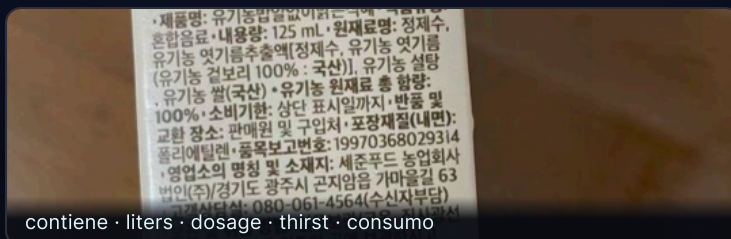
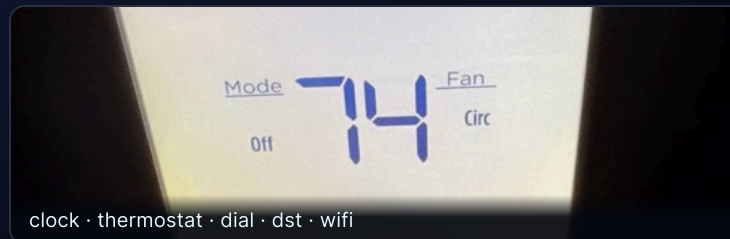
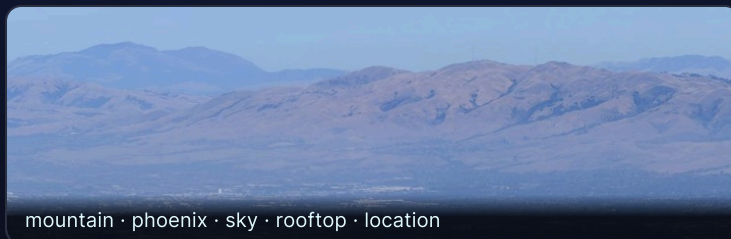
在**同一个任务**上，推理算力很快就见顶了。
那如果我们不要求它做得更好，而是要求它做一件**全新的事**呢？

03 | 实验二 · 用算力换新功能

同样是冻结的向量模型，这次要它做一件训练时根本不存在的事

— 先看结果

一个只会做检索的模型，直接吐出了标签。



没有训练、没有第二个模型、没有词典。这些标签全部来自推理期计算。

问题设定

和实验一同样严的约束，但换了一个目标。

- | **GIVEN** 一个向量模型 **jina-embeddings-v5-omni-nano** (~1B, 图文共享 768 维空间)，一张图。
- | **OUTPUT** 图中**每一个**物体的词：多标签、开放词表。
- | **FROZEN** 全部权重冻结。它是检索编码器，没有标注头、没有分类器，从未为此任务训练过。
- | **ALLOWED** 只允许在模型自身输出上做推理期计算，别的都不行。

不训练

不引入第二个模型

不用 WordNet / 词典

不用正则 / 词性标注

注意这次的差别：实验一是把检索做得更准，实验二是让检索模型去做它压根不会的事。

对向量模型来说，test-time compute 只有三种花法。

A

同一次前向，多做代数

不再只读池化向量，改读 patch 级输出，和 128k 词表逐一打分。计算量几乎不变。

对应实验一里的 $c = 1.0$

B

新的视角，新的前向

把图切成多个 crop 重新编码，小物体在自己的 crop 里占满画面。计算量线性增长。

对应实验一里的 $c > 1$

C

对已有特征再加工

白化、最优传输、图传播、EM……在**同一批像素**上做更重的数学。看起来最「聪明」。

实验一里塌掉的那一类

结论提前说：A 有效，B 有效，C 是幻觉。
和实验一的结论惊人地一致。

方法

第四步，把检索器改造成标注器。

tokenizer 词表 → 标签空间



patch × 128,260 打分



减去背景先验 $\mu\ell$



词首 gate + embedding-NMS

01 词表就是开放标签空间

128,260 个 token 全部编码成文本向量。不需要人工标签集，模型认识多少词就有多少候选。

02 每个 patch 单独打分，再与全局向量融合

$0.7 \times \text{patch-max} + 0.3 \times \text{global}$ 。小物体不再被整图池化稀释——这是 mAP 从 0.264 到 0.635 的来源。

03 减去每个 token 自己的先验

高频 token 天然离所有图都近。减掉 $\mu\ell$ 之后，右尾才是真正的证据。

04 词首 gate 与向量空间 NMS

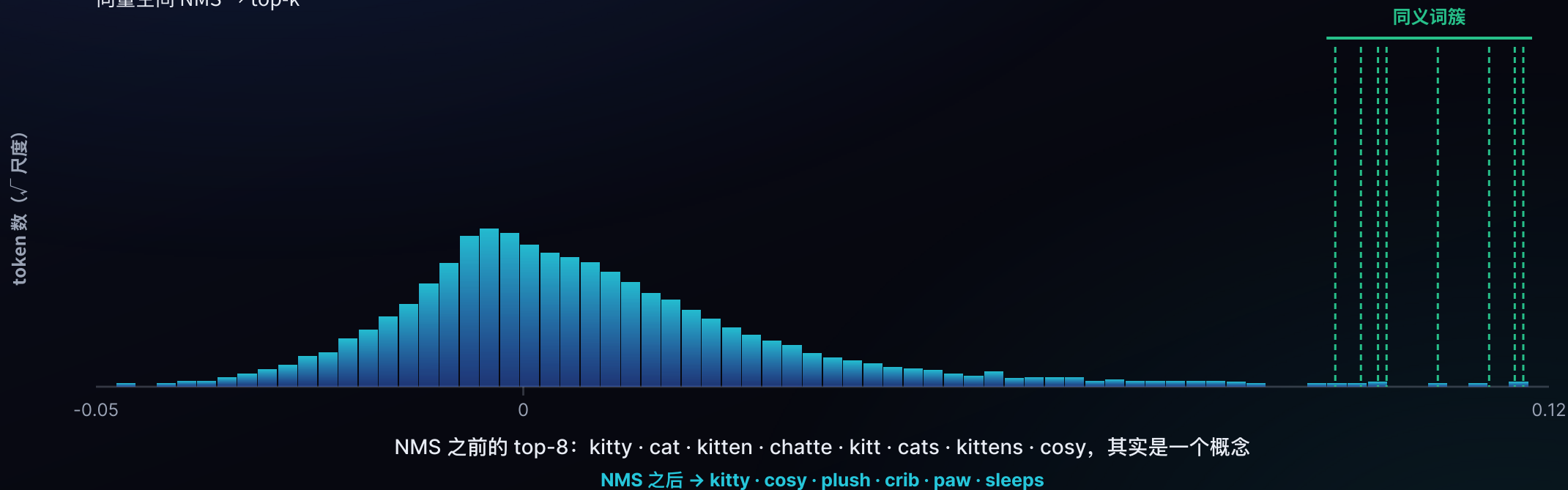
128,260 → 25,465 候选；再把 kitty/cat/kitten/chatte 这样的同义簇压成一个概念。

真实分布

12.8 万 token 的得分分布，一步步被整形。

4 · 右尾去重

向量空间 NMS → top-k



COCO-150

精度是被「读得更多」和「看得更多」推上去的。



METHOD	P@1	P@3	R@5	MAP
global pooled	0.433	0.289	0.449	0.264
patch max + global	0.753	0.427	0.631	0.635
softpool	0.773	0.449	0.649	0.608
+ CWR multi-crop	0.813	0.476	0.680	0.710

全局池化 0.264 → patch 融合 0.635 → 多 crop 重编码 0.710。

A 类贡献 +0.371，B 类再贡献 +0.075。

全部杠杆

2024-2026 的免训练方法，我们挨个跑了一遍。



算力—精度

只有「新像素」能换到精度，「更重的数学」换不到。



C 类不是「算得不够多」，是假设本身不成立：图文本来就在一个空间里，分数早已校准，没有额外信息可挖。

研究直接变成产品：本机全盘跨模态搜索。

~4%

标注开销：搭在原有 embedding 前向上顺带完成

0.847

HQ 重跑 (5-crop CWR) P@1, Swift/bf16 端口, 基线 0.773

32 帧

视频：每 240s 片段采样，跨空间与时间取 patch-max

图片

同一个标签矩阵，索引时顺带产出标签，无需二次前向。

视频

帧采样后共用同一套 patch 打分逻辑，时间维直接并入 max。

扫描版 PDF

页面当图处理，OCR 不可用的老档案也能被词检索到。

把两个实验接起来

同一条规律，在两个完全不同的任务上出现了两次。

01

换精度：很快见顶

任务没变，模型已经在这个任务上被训练到位了。多花的算力大多在重新表述同一批信息，域内好看，域外崩。

真正留下来的是 $c = 1.0$ 到 1.5 的那一批廉价结构。

02

换新功能：真的买到了东西

任务是新的，模型的几何里有从没有被读出来过的信息。

读 patch (A) 拿到 $+0.371$ ，重新编码新像素 (B) 再拿 $+0.075$ 。

有效的算力

让**新信息**进入系统：被池化丢掉的 patch，被缩放丢掉的像素，被单向量丢掉的句子结构。

无效的算力

在**同一批信息**上做更漂亮的数学：白化、最优传输、图传播、EM、可学习 head。

— 结论

搜索就是 test-time compute。能力不在权重里，在你愿意为一次推理花多少算力上。

—— 但只有换来新信息的算力才算数

01 冻结的小模型里，藏着的东西比你以为的多

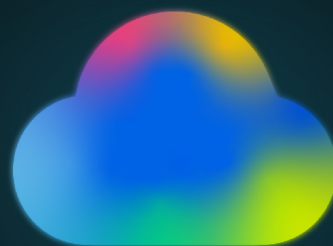
一个 239M 的编码器能被重组出经典 IR 的全套结构；一个 1B 的检索模型里已经存在一个开放词表标注器。

02 但算力不是万能药，它只是一个搬运工

它能把已有的信息搬到你能用的地方。它搬不来本来就不存在的信息。

03 所以先问「这笔算力带来了什么新信息」，再问「花多少」

没有新信息进来的算力，不管数学多漂亮，域内多好看，都换不到真正的东西。



THANKS

肖涵 Han Xiao · Elastic AI 副总裁 · @hxiao · arXiv:2605.11374

GITHUB REPO

